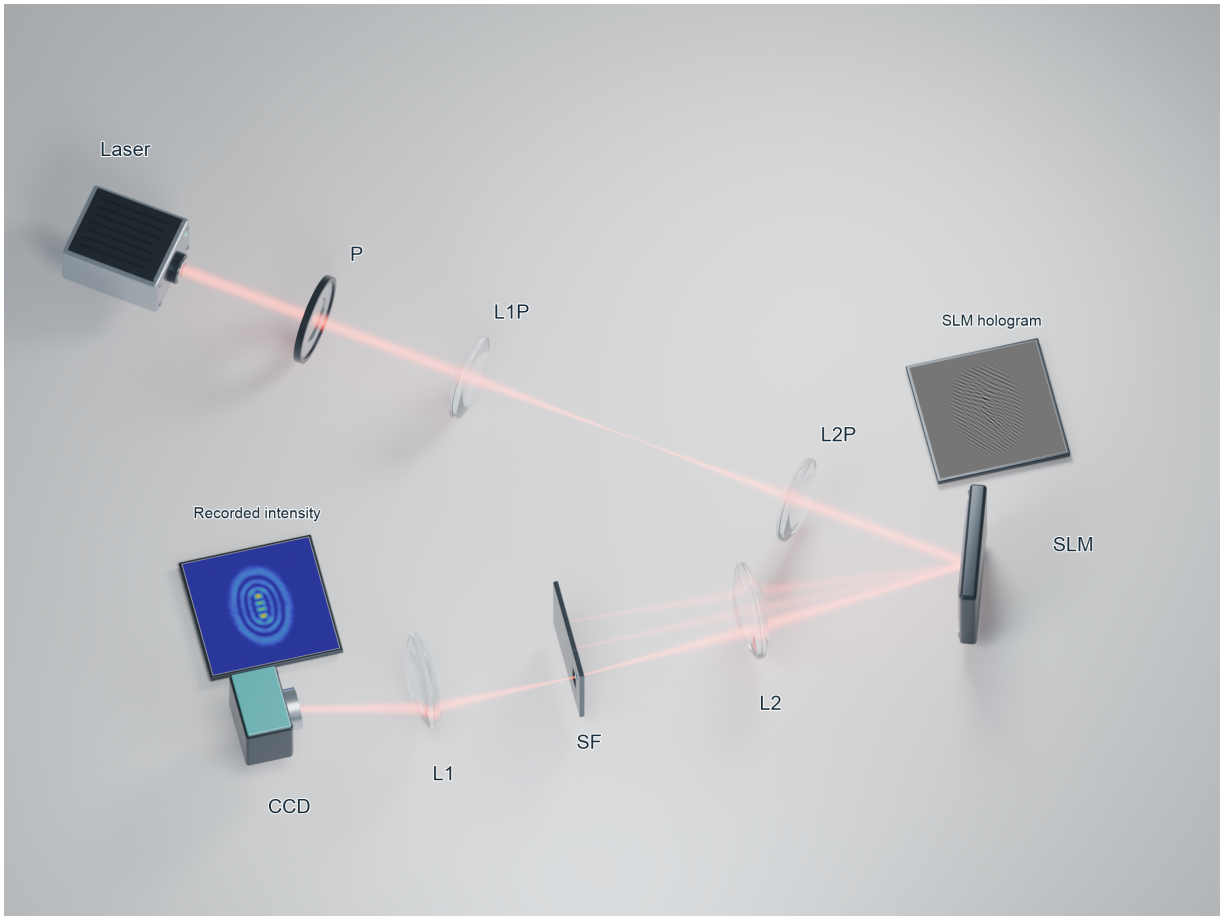
Blender Optical Schematics

From a fresh download to your first figure

B. M. Rodriguez-Lara

10 November 2026



What you will make. A Blender recreation of the optical setup illustrated in the paper, with an editable Blender file, two rendered views and SVGs with editable labels. This image is a reference preview from our later reconstruction, with the supplied SLM hologram and CCD intensity panels beside their instruments. Your first run follows the supplied layout; exact framing may vary because the agent writes the scene builder.

What inspired this project. The basic optical-setup schematic in https://doi.org/10.1103/6ftp-svh8 motivated the development of this Blender library. The paper contains no Blender model. The models and this example were created afterward; they were neither used in nor supplied with that publication. The paper DOI identifies the article, not this library.

No Blender coding experience is assumed. Follow the steps in order. The catalogue image is available separately at `images/catalogue-cover.png`.

1. Install the two applications

1. Install Blender from https://www.blender.org/download/. These assets were prepared with Blender 5.2.3 LTS. Use that version or a compatible newer version; older versions have not been tested.

2. Launch Blender once to finish its initial setup. Confirm that its normal scene window opens. You can close it afterward.

3. Install/open Codex from the official OpenAI application documentation, https://developers.openai.com/codex/app/, and sign in. Use a local project session that can read files and run Blender. A browser-only conversation without local file/execution access cannot perform this workflow on your computer.

Roles. Blender builds and renders the scene. Codex reads the agent instructions and writes/runs the build script. The supplied agent is a folder of instructions, not a third application or a Blender add-on. No API programming is needed for this walkthrough.

2. Download and keep the complete project together

1. If using GitHub, open this project's repository, choose Code, then Download ZIP. Extract the ZIP. Do not work inside the compressed ZIP. If you received a folder directly, use that complete folder.

2. Put the extracted project in a stable location, for example `Documents/OpticalElements/blender-optical-schematics/`. You may rename a downloaded folder ending in `-main` to `blender-optical-schematics`.

3. Open that folder in your file browser. You should see the following structure. This folder is called the **project root** in the remaining steps.

```
blender-optical-schematics/
  README.md
  library/                  Blender asset files
  agent/                    The agent's own folder
    README.md
    install.py
    setup-to-blender/
      SKILL.md
      agents/
      references/
  examples/paper-setup/      Prompt and starting layout
  scripts/                  Reusable beam helper
  images/                   Catalogue and example preview
  docs/                     This PDF and its LaTeX source
```

Checkpoint. Keep `library/` and `agent/` here. Do not move them into Blender's application folder or split them across unrelated locations. All later relative paths start at this project root.

3. Register the library in Blender

These steps let you browse and drag components by hand. The agent's build script can also read the same library files directly.

1. Open Blender and open Preferences (normally Edit, then Preferences; on some macOS versions use the Blender application menu).
2. Find Asset Libraries. Depending on Blender version it is a separate preferences section or appears under File Paths.
3. Click the plus/add button. Browse to the project root and select its `library` folder. Select the folder, not an individual `.blend` file and not the `agent` folder.
4. Give it a recognizable name such as `Optical Schematics`. Save preferences if automatic saving is disabled.
5. In a Blender editor area's upper-left editor-type menu, choose Asset Browser. Select `Optical Schematics` in its library selector. If categories are empty, select All to see every asset.
6. Drag one collection into a fresh scene to confirm that it appears. You can discard this test scene. Do not save changes over a file inside `library/`.

Checkpoint. You can see optical components in the Asset Browser. Schematic assets use 1 Blender unit = 10 mm, with illustrative geometry. Catalogue thumbnails normalize sizes for legibility; use the source collection scale in your scene.

4. Open the correct folder in Codex

1. Use Codex's project/folder selection to open the `blender-optical-schematics` folder you saved in Step 2. Choose the project root, not `library/`, `agent/` or the parent folder containing both optical projects.
2. Start a chat attached to that local folder. Select local work if your interface offers a choice between local and remote execution.
3. Paste this readiness check. It does not ask Codex to generate the figure yet.

```
Check that this project contains library/, agent/, and
examples/paper-setup/. Locate Blender on this computer and
report its version. Do not generate a figure yet. If any
required file or application is missing, tell me exactly
what I need to fix before continuing.
```

Checkpoint. Codex confirms the three folders and finds Blender. If Blender is not on the system search path, give its executable location. On macOS it is commonly `/Applications/Blender.app/Contents/MacOS/Blender`. On Windows, locate `blender.exe` inside the installed Blender folder; the versioned folder name varies.

5. Put the agent in place

Recommended first run. Leave the complete `agent/` folder where it is. No extra installation is required when you call it by file path in Step 6. This avoids hidden-folder setup and automatic-discovery issues.

Optional named call. If you want to type `$setup-to-blender`, install a project-local copy. In Codex, paste:

```
Install this project's agent by running agent/install.py
with an available Python interpreter from this project
root. Do not install globally or overwrite a different
existing agent. Report the installed SKILL.md path.
```

The installer uses only standard Python. If Python is unavailable, the manual alternative is to copy the whole `agent/setup-to-blender` folder into `.agents/skills/` under this same project. Codex can perform that copy if you ask. The installed path must be:

```
blender-optical-schematics/.agents/skills/setup-to-blender/SKILL.md
```

The dot-prefixed `.agents` folder is normally hidden. Keep the visible `agent/` folder as the original distribution. After optional installation, start a new chat in the same project. If the named skill is still not recognized, use the direct call below; it does not depend on skill discovery.

6. Generate the example inspired by the paper

For the first run, use the supplied list and layout without adding files. Paste this exact request in the project chat:

```
Read agent/setup-to-blender/SKILL.md and follow it to carry
out examples/paper-setup/prompt.md. Use this project only.
Generate the paper-setup example now, not just a plan.
Use the supplied layout assumptions and both bundled
SLM/CCD image panels. Save to a new dated Figures folder; preserve
existing files and libraries. Run Blender, inspect both
views, and give me links to the Blender file, renders,
paired SVGs and the validation/assumptions notes.
```

If the named skill is installed, the shorter equivalent is:

```
Use $setup-to-blender to execute
examples/paper-setup/prompt.md and generate the example.
```

Codex writes the scene plan and build script, runs Blender, exports the figures and checks them. Rendering can take time; wait for a completion report. Respond to any application access request only after checking that it concerns this project and Blender. Do not close the running task before it finishes.

7. Find and open your finished figure

The agent selects an unused figure number using your local date. A typical output folder looks like this; the date and filenames may differ, so use the links in its completion message.

```
Figures/YYYY-MM-DD-FIgNN/
  build.py                  Repeatable Blender build
  setup.json                Interpreted scene plan
  ...blend                  Editable Blender scene
  renders/                  Perspective and plan PNGs
  svg/                      Lettered/unlettered SVG pairs
  README.md                 Assumptions and checks
```

1. Double-click the `.blend` file to open it in Blender. It is the editable result, not a library source file.
2. Open the PNGs in `renders/` to review the perspective and overhead views. A window showing only a default cube is not a completed result.
3. Open a lettered SVG in a vector editor to edit text. The optics are an embedded raster render; the lettering is vector text. The unlettered SVG omits the lettering layer. Default output width is 86 mm.
4. Read the figure README. Check the optical sequence, labels, assumptions and reported validation before using the image in a publication.

Expected topology. Laser, circular pupil P, L1P, L2P, reflective SLM, L2, filter SF, L1, CCD. The selected +1 order continues through SF; rejected orders end there. P is not a polarizer. No extra beam splitter is implied. The documented figure labels put L2 nearest SLM and L1 nearest CCD; earlier notes report conflicting manuscript prose.

What is assumed. Positions and mounting choices are authored layout assumptions, not measurements. Focal lengths are unknown. The experimental pupil and filter openings are assumed to be 4 mm and 1 mm. Gaussian envelopes are visual guides, not diffraction simulations. Mechanical fit is not certified.

8. Add your own sketch or data images later

Create `inputs/paper-setup/` inside the project and place your lawful sketch/PDF and any SLM/CCD reference PNGs there. Tell Codex the exact filenames and which component each image belongs to. Ask for a new figure variant rather than overwriting the first run. The example includes two PNGs in `examples/paper-setup/reference-images/`: `slm_hologram.png` beside SLM and `ccd_intensity.png` beside CCD. They are supplied reference imagery, not Blender simulation results. Their source and existing rights are recorded in that folder's README. The full paper PDF is not bundled. Without a supplied sketch the agent reconstructs the documented topology and must not claim to have checked the original drawing.

9. If something does not work

Codex cannot find SKILL.md or layout.json.
You probably opened the wrong folder or did not extract the ZIP. Return to Step 2, then open the project root in Codex. Confirm `agent/setup-to-blender/SKILL.md` exists.

The agent is not listed.
Use the direct file-path call in Step 6. If using the optional installer, verify the exact hidden-folder path in Step 5 and start a new project chat.

The Blender Asset Browser is empty.
Confirm you registered the `library/` folder containing the `.blend` files. Select the correct library and All categories. Reopen Blender if needed. Registration affects manual browsing; a script can still append directly from existing library files.

Blender is not found.
Install and launch it once. Give Codex its actual executable path. Do not supply a shortcut or the containing folder when an executable is requested.

Python or Pillow is missing.
Ask Codex to identify the missing dependency and use a project-local environment for the exporter. Blender has its own Python for scene builds. The optional installer needs standard Python, but the direct file-path agent call does not need the installer.

Codex only returns a plan.
Paste: "Continue and generate the files. Run Blender, export both views and paired SVGs, then inspect and link the results." If it cannot execute locally, switch to a local project session or follow its reported installation fix.

Rendering is slow.
Ask for a separate preview at 1000 by 800 pixels and 12 samples. Once it looks right, request the final 2000 by 1600 images with 40 samples. Do not present the preview as the final render.

An SLM or CCD panel is missing.
Confirm both bundled reference PNGs exist. Ask the agent to load them, pack the textures and inspect both camera views. Do not substitute invented data.

Labels, apertures or beams look wrong.
Ask for a corrected variant and state the specific discrepancy. Moving an optic does not automatically reroute its beam; both need updating in the scene plan. Preserve the original result for comparison.

Sources, licenses and editable documentation

The agent installation convention is documented at https://developers.openai.com/codex/skills/. Blender's library preferences are documented at https://docs.blender.org/manual/en/5.4/editors/preferences/asset_libraries.html. Menu placement varies by version.

Original code uses MIT; original authored models and documentation use CC BY 4.0. See LICENSE and `docs/provenance.md` for exclusions. See the following section for the library DOI and citation formats.

This guide's editable source is `docs/user-guide.tex`. To rebuild it, run `pdflatex user-guide.tex` twice from `docs/`. The example image is `images/paper-example.png`. The full catalogue is separate; there is no collection index in this guide.

10. How to cite

When using this library to prepare a figure, cite the project using its stable DOI:

Rodriguez-Lara, B. M. (2026). *Blender Optical Schematics* [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.23993947

Stable project DOI. This DOI identifies Blender Optical Schematics across all releases and opens the latest archived version. It remains the same as the library develops.

BibTeX for LaTeX

Copy this entry into your bibliography `.bib` file, or use the supplied `docs/citations/schematics-project.bib`. The `@misc` entry works with traditional BibTeX styles; the note identifies the work as computer software.

```
@misc{rodriguez_lara_2026_optical_schematics,
  author    = {Rodriguez-Lara, B. M.},
  title     = {{Blender Optical Schematics}},
  year      = {2026},
  publisher = {Zenodo},
  doi       = {10.5281/zenodo.23993947},
  url       = {https://doi.org/10.5281/zenodo.23993947},
  note      = {Computer software}
}
```

In your manuscript, use `\cite{rodriguez_lara_2026_optical_schematics}` and compile with your normal bibliography workflow.

Other formats and reference managers

- **RIS:** import `docs/citations/schematics-project.ris` into a reference manager such as Zotero or EndNote.
- **CSL-JSON:** use `docs/citations/schematics-project.csl.json` with tools supporting Citation Style Language data.
- **Plain text:** copy `docs/citations/schematics-project.txt` and adapt the formatting to your journal's style.

Cite the library for the Blender assets. Use the library DOI above when using these models. Cite the paper at https://doi.org/10.1103/6ftp-svh8 only when referring to its optical setup or scientific results. Its basic schematic inspired this project; the article contains no Blender model. The models were created afterward and were neither used in nor supplied with the publication. Library citation does not replace the attribution required by CC BY 4.0 for reused models, figures and documentation.

Reusable POV and propagation accessories

The named preset **Folded Relay — Front-Left** preserves the accepted orthographic viewing direction. It is stored in `library/camera-presets/folded-relay-front-left.json`. It records position, rotation, target, orthographic scale, clipping planes, resolution and scene units. The reference position is $(-34, -20, 40)$, with target $(-3, -1, 1.6)$ and orthographic scale 36 in a scene using 0.01 metres per Blender unit.

In Blender's Python Console, replace the project path and run:

```
import sys
sys.path.insert(0, '/path/to/project/scripts')
from camera_presets import apply_camera_preset
apply_camera_preset(bpy.context.scene)
```

The helper converts reference distances to the active scene's units. Use `unit_scale=1` to replay the coordinates literally. For a table scene, retain the same direction and adjust the framing:

```
apply_camera_preset(bpy.context.scene,
    target=(-0.08, -0.02, 0.04),
    ortho_scale=0.90, resolution=(2400, 1900))
```

The target and orthographic scale use destination Blender units. Rebuild camera-facing panels after applying a different camera.

The new collections are:

- `SO_Translation_Rail`: Fixed translation rail; use with independent carriage.
- `SO_Translation_Carriage`: Moving carriage; use the same translation transform for the detector and its support.
- `SO_Quadratic_Beam_Envelope`: Translucent quadratic-width beam visualization. Not a diffraction calculation.

Append these collections and use collection instances. Translate the carriage, detector and detector support together. Keep focal lengths and scene-specific apertures in figure metadata; generic lenses remain shape assets.

The quadratic beam envelope is a geometric illustration, not simulated diffraction. The procedural helper is `scripts/beam_envelope.py`. Rejected orders retain the same red hue at reduced brightness. No cable connections or direction arrows are added by default.

For an experimental scene, reuse the existing generic laptop. Align its long edge parallel to the chosen table edge and place supplied CCD and SLM images on its existing display planes. Keep private reference images in the figure source directory, not in the reusable asset files.

Export matching lettered and unlettered SVGs at the requested width (86 mm for one column), with explicit millimetre dimensions and 0.5 mm borders when requested. The optical render is embedded raster; lettering is a separate editable vector layer. See `docs/propagation-and-camera.md` for details.

Optional: search for a compact camera view

If a schematic contains too much empty space, ask the agent to compare camera viewpoints. Keep an accepted POV unless you want alternatives. The experimental project's accepted viewpoint is independent of this option.

Copy this request into Codex:

> Search for a more compact orthographic POV for this schematic. Keep optical geometry, component instances and focal spans fixed. Preserve the original figure. Compare a coarse grid of azimuths and elevations, then refine promising views. Re-anchor the camera-facing CCD and SLM panels at every angle. Rank complete views by approximate empty page area at 86 mm width, but reject views that obscure optical faces, merge beam branches or leave insufficient room for readable labels. Show a comparison, save the selected camera as a reusable preset, and export lettered and unlettered SVGs with 0.5 mm borders. Close application windows opened for the task when finished.

How the search works

1. Save a separate figure directory. Keep component geometry fixed and render transparent previews with consistent camera scale, resolution and lighting. Re-anchor reference panels for every camera angle.

2. Compare azimuths and elevations. The reusable Blender helper is `scripts/search_camera_views.py`; it restores the original camera, object transforms and render settings after the sweep. A panel-update callback must be supplied for scenes with reference panels.

3. Run the Pillow-based comparison helper outside Blender:

   ```
   python scripts/rank_camera_views.py /path/to/candidates
   ```

   It produces `comparison.png` and `ranked.json`. The metrics are cropped page height, alpha-mask occupied fraction and approximate empty page area at the requested publication width. Empty or clipped renders are flagged as invalid.

4. Inspect readability before selecting a winner. Check iris and SLM faces, component separation, beam branches, image panels and final lettering. Refine the angle grid around promising candidates. A lower numerical score does not override a readability failure.

5. Apply the chosen camera JSON through `apply_camera_preset`, re-anchor panels, render at final quality and export the paired SVGs. Keep the original figure and preset.

Included compact preset `library/camera-presets/folded-relay-compact-oblique.json` stores **Folded Relay — Compact Oblique**, with azimuth $-75°$ and elevation $25°$. Azimuth runs from world $+X$ toward $+Y$; elevation is above the XY plane. In the initial 15-view study, this gave a readable 86×23.47 mm figure, compared with an earlier height of 80.58 mm. These values describe that layout, not a universal or proven global optimum.

Alpha coverage is a proxy that depends on the threshold for glass and translucent beams. Keep all four requested borders, preserve render aspect ratio and check the final lettering. The optical render remains raster inside the SVG; text stays editable. Full examples and parameter meanings are in `docs/camera-view-search.md`.